Chapter 14: Inheritance

ENG3091

Khiem Nguyen

Contents

1 Introduction to inheritance **2**
- 1.1 Shapes: "is a" and "has a" . . . 2
- 1.2 Hierarchies . . . 2
- 1.3 The people of a university . . . 3
- 1.4 A better design . . . 4

2 Basic inheritance in C++ **4**
- 2.1 Terminology . . . 4
- 2.2 A smaller example for our code . . . 5
- 2.3 The class `UniPerson` . . . 6
- 2.4 `Lecturer` without inheritance . . . 7
- 2.5 `Lecturer` inherits from `UniPerson` . . . 8
- 2.6 Using `Lecturer` . . . 8
- 2.7 Adding `Student` is now easy . . . 9
- 2.8 `Lecturer` and `Student` together . . . 9

3 Splitting the program into files **10**
- 3.1 Why split, and how . . . 10
- 3.2 `UniPerson`: the header . . . 10
- 3.3 `UniPerson`: the implementation . . . 11
- 3.4 `Lecturer`: header and implementation . . . 11
- 3.5 `Student`: header and implementation . . . 12
- 3.6 `UGStudent`: an inheritance chain . . . 12
- 3.7 The main program, and building it . . . 13

4 Order of construction **14**
- 4.1 A base class and a derived class . . . 14
- 4.2 A derived object has two parts . . . 14
- 4.3 The base part is built first . . . 15
- 4.4 Inheritance chains . . . 15
- 4.5 Construction order in a chain . . . 16

5 Constructing derived classes **17**
- 5.1 Coming from Python: `super()` . . . 17
- 5.2 The problem . . . 17
- 5.3 What happens when a `Derived` is created . . . 18
- 5.4 Attempt 1: initialize `id` in `Derived`'s list . . . 18
- 5.5 Attempt 2: assign `id` in the body . . . 19
- 5.6 The solution: call the base constructor . . . 20
- 5.7 Passing values up a chain . . . 21
- 5.8 Destruction: the reverse order . . . 22

6 Inheritance and access specifiers **22**
6.1 The `protected` access specifier . . . 22
6.2 Access specifiers and derived classes . . . 23
6.3 When should we use `protected`? . . . 23
6.4 Kinds of inheritance (read at home) . . . 24

7 Adding, redefining and extending behaviour **24**
7.1 Adding new functionality . . . 24
7.2 Using the new functionality . . . 25
7.3 A vector of derived objects . . . 25
7.4 Calling a base class function . . . 26
7.5 Redefining behaviour . . . 26
7.6 Adding to existing behaviour . . . 27
7.7 Changing access with `using` (read at home) . . . 28

8 Multiple inheritance **28**
8.1 Multiple inheritance . . . 28
8.2 A smartphone in code . . . 29

9 Wrapping up **29**
9.1 Summary . . . 29
9.2 Keep learning . . . 30

1 Introduction to inheritance

1.1 Shapes: "is a" and "has a"

You're building software that draws 2D shapes: rectangles, squares, polygons, circles, ellipses… One class per shape? Wait a minute:

"Is a" relationships:

- a `Square` **is a** `Rectangle`;
- a `Rectangle` **is a** `Polygon`;
- a `Circle` **is an** `Ellipse`;
- all of them **are** `Shapes`.

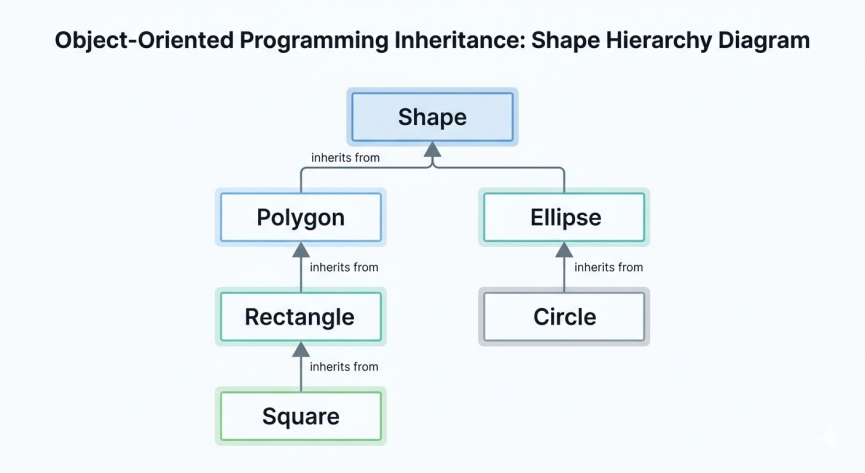
"Has a" relationships:

- a `Polygon` **has a** list of `Point2Ds`;
- an `Ellipse` **has a** centre (a `Point2D`) and two axis lengths `a` and `b`;
- a `Circle` is an `Ellipse` with `a == b`.

So why build all those classes separately? Drawing a square should just be drawing a rectangle with equal sides.

1.2 Hierarchies

A **hierarchy** is a diagram showing how things are related.



> ⚠ Watch out
>
> Everyday "is a" can mislead. A square *is* a rectangle, but if `Rectangle` has `set_width()`, a `Square` can't honour it without breaking its own rules. This classic trap is the *square–rectangle problem*. In code, "is a" must mean "can be used **anywhere** the base class is used".

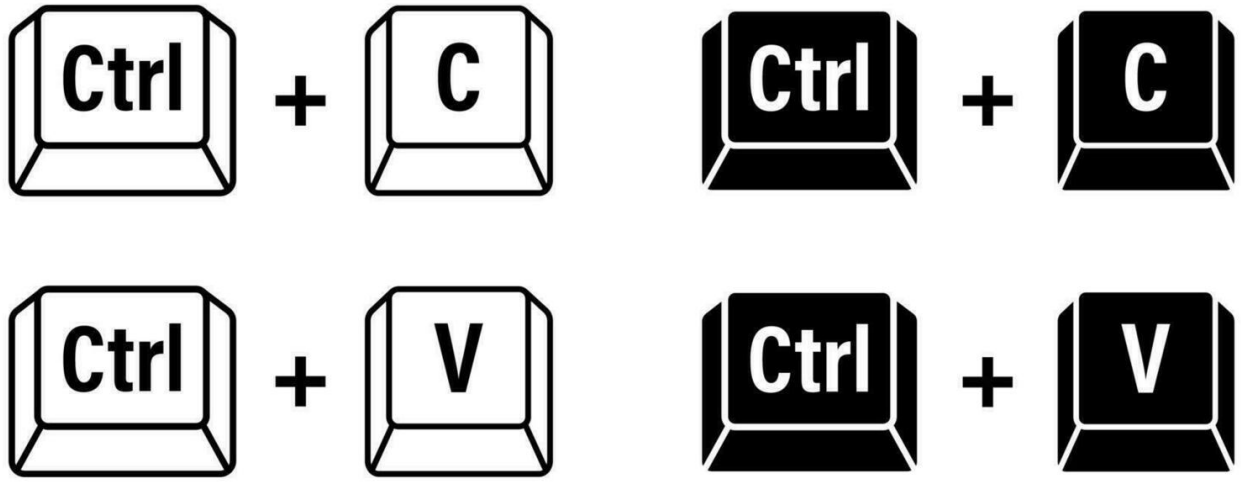
1.3 The people of a university

If shapes don't convince you, look at our university:

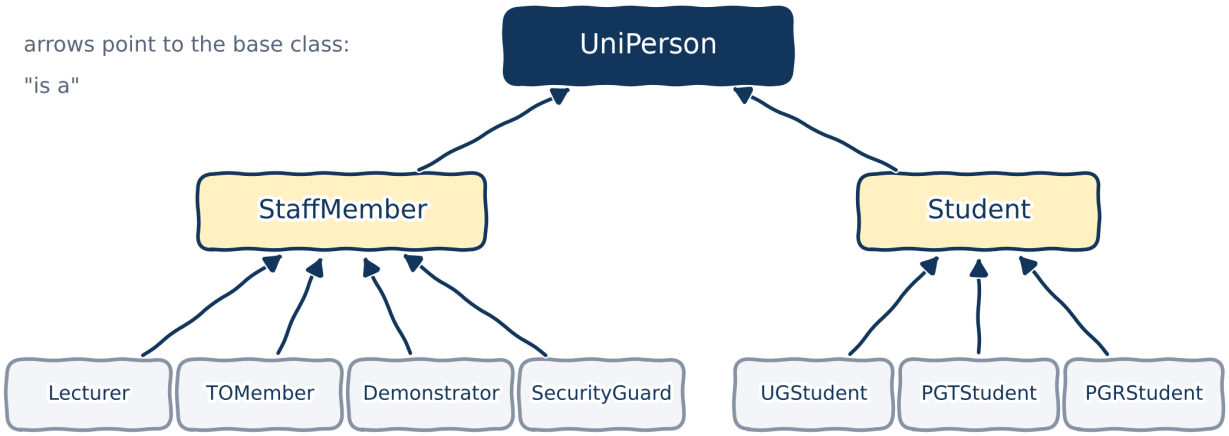
- every **staff member** has a `name`, a `birthday` and an `ID`;
- every **student** has a `name`, a `birthday` and an `ID`;
- there are different kinds of staff: `Lecturer`, `TOMember`, `Demonstrator`, `SecurityGuard`;
- and different kinds of students: `UGStudent`, `PGTStudent` (postgraduate taught), `PGRStudent` (postgraduate research).

Writing all seven classes separately means writing the common members seven times: copy, paste, repeat.

That breaks the golden rule from Chapter 02: **Don't Repeat Yourself**.



1.4 A better design



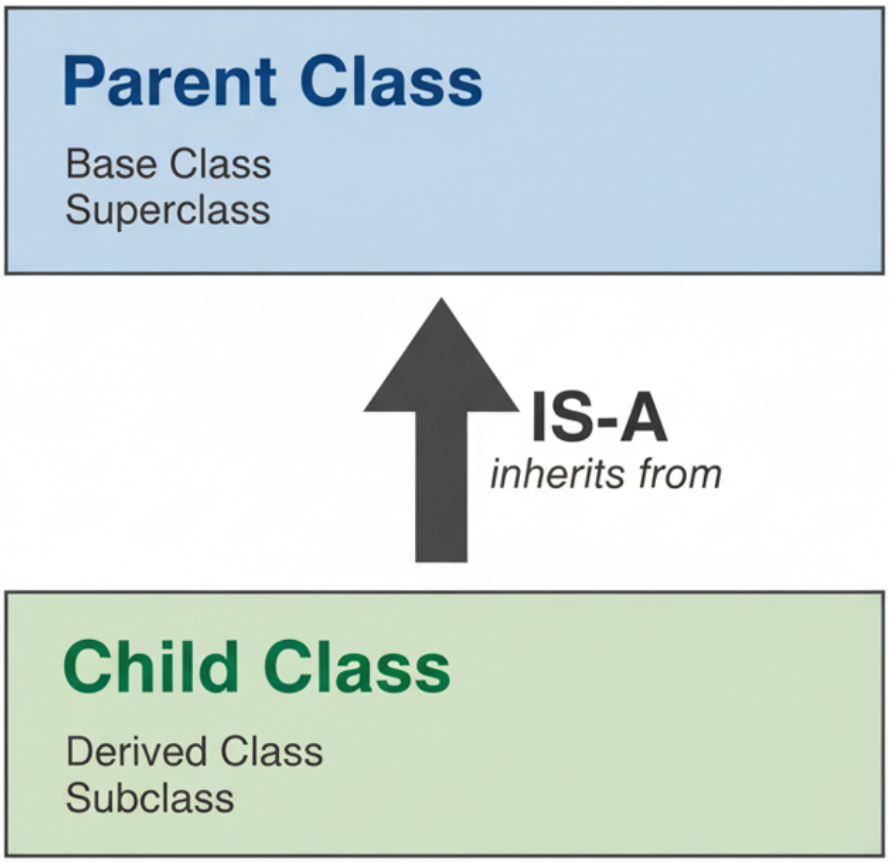
What all people share goes in `UniPerson`; what all staff share goes in `StaffMember`; and each role adds only what's special about it.

2 Basic inheritance in C++

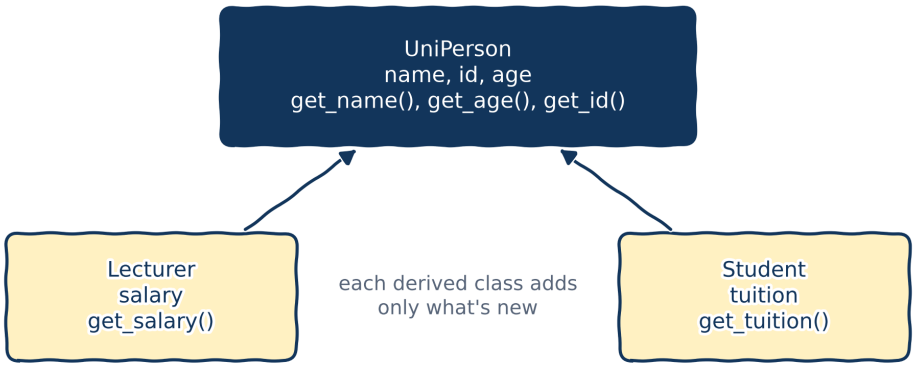
2.1 Terminology

In an inheritance ("is a") relationship:

- the class being inherited from is the **base class** (or *parent class*, or *superclass*);
- the class doing the inheriting is the **derived class** (or *child class*, or *subclass*).



2.2 A smaller example for our code



Every `Lecturer` and every `Student` **is a** `UniPerson`, with a name, an ID and an age. What they *do* differently comes with polymorphism, in Chapter 15.

2.3 The class `UniPerson`

Listing 1 `uniperson.h`

```h
#ifndef UNIPERSON_H
#define UNIPERSON_H

#include <string>

class UniPerson {
  private:
    static inline int next_id{100000}; // ONE counter, shared by all
    std::string name{};
    int id{};
    int age{};

  public:
    UniPerson(const std::string& name = "", int age = 0)
        : name{name}, id{next_id++}, age{age} {}

    const std::string& get_name() const { return name; }
    int get_age() const { return age; }
    int get_id() const { return id; }
};

#endif
```

A **`static`** member variable belongs to the **class**, not to any one object: all `UniPerson`s share one `next_id`. Each new person takes its value, then increments it.

`inline` (C++17) lets us initialize it right here, in the header.

2.4 `Lecturer` without inheritance

Listing 2 `lecturer_noinheritance.cpp`

```cpp
#include <string>

class Lecturer { // no inheritance: everything written again
  private:
    static inline int next_id{100000}; // a SECOND, separate counter!
    std::string name{};
    int id{};
    int age{};
    double salary{};

  public:
    Lecturer(const std::string& name, int age, double salary)
        : name{name}, id{next_id++}, age{age}, salary{salary} {}

    const std::string& get_name() const { return name; } // copied...
    int get_age() const { return age; }                  // ...and
    int get_id() const { return id; }                    // pasted
    double get_salary() const { return salary; }
};

int main() {
    Lecturer khiem{"Khiem", 42, 55000};
    return 0;
}
```

It compiles. So what's the problem?

- Everything from `UniPerson` is **copied and pasted**.
- `Lecturer` has its **own** `next_id`, so lecturer and student IDs can clash.
- Fix a bug in one copy, and you must remember all the others.

2.5 `Lecturer` inherits from `UniPerson`

Listing 3 `lecturer.h`

```
#ifndef LECTURER_H
#define LECTURER_H

#include "uniperson.h"
#include <string>

class Lecturer : public UniPerson { // a Lecturer IS a UniPerson
  private:
    double salary{}; // only what's new

  public:
    Lecturer(const std::string& name = "", int age = 0, double salary = 42000)
        : UniPerson{name, age}, salary{salary} {}

    double get_salary() const { return salary; }
};

#endif
```

- `class Lecturer : public UniPerson` means "a **`Lecturer` is a `UniPerson`**". We'll see what `public` means here later today.
- `Lecturer` only adds what's new: `salary`. The member initializer list first passes `name` and `age` to `UniPerson`'s constructor (more on this soon).

2.6 Using `Lecturer`

Listing 4 `main_uniperson_lecturer.cpp`

```
#include "lecturer.h"
#include <iostream>

int main() {
    Lecturer khiem{"Khiem", 42};
    std::cout << khiem.get_name() << ", age " << khiem.get_age() << ", ID "
              << khiem.get_id() << '\n';

    Lecturer albus{"Albus", 115}; // a very experienced lecturer
    std::cout << albus.get_name() << ", age " << albus.get_age() << ", ID "
              << albus.get_id() << '\n';
    return 0;
}
```

```
Khiem, age 42, ID 100000
Albus, age 115, ID 100001
```

We never wrote `get_age()` or `get_id()` in `Lecturer`, yet we can call them: `Lecturer` inherited them from `UniPerson`.

2.7 Adding `Student` is now easy

Listing 5 `student.h`

```cpp
#ifndef STUDENT_H
#define STUDENT_H

#include "uniperson.h"
#include <string>

class Student : public UniPerson { // a Student IS a UniPerson
  private:
    double tuition{};

  public:
    Student(const std::string& name = "", int age = 0, double tuition = 9250)
        : UniPerson{name, age}, tuition{tuition} {}

    double get_tuition() const { return tuition; }
};

#endif
```

2.8 `Lecturer` and `Student` together

Listing 6 `main_uniperson_lecturer_student.cpp`

```cpp
#include "lecturer.h"
#include "student.h"
#include <iostream>

int main() {
    Lecturer khiem{"Khiem", 42, 55000};
    std::cout << khiem.get_name() << " (ID " << khiem.get_id() << "), salary "
              << khiem.get_salary() << '\n';

    Student peter{"Peter Parker", 18};
    std::cout << peter.get_name() << " (ID " << peter.get_id() << "), tuition "
              << peter.get_tuition() << '\n';
    return 0;
}
```

```
Khiem (ID 100000), salary 55000
Peter Parker (ID 100001), tuition 9250
```

One shared counter: IDs never clash, whether you're a lecturer or a student.

3 Splitting the program into files

3.1 Why split, and how

A real program has many classes. As in Chapter 02, each class gets:

- a **header file** (`.h`), with the class definition and its member function *declarations*;
- an **implementation file** (`.cpp`), with the member function *definitions*.

We'll build four classes and a `main` program, in the folder `uni-program/`:

- `UniPerson`;
- `Lecturer` and `Student`, which inherit from `UniPerson`;
- `UGStudent`, which inherits from `Student`: an **inheritance chain**.

3.2 `UniPerson`: the header

Listing 7 `uni-program/uniperson.h`

```
#ifndef UNIPERSON_H
#define UNIPERSON_H

#include <string>

class UniPerson {
  private:
    static int next_id; // declared here, defined in uniperson.cpp
    std::string name{};
    int id{};
    int age{};

  public:
    UniPerson(const std::string& name = "", int age = 0);
    const std::string& get_name() const;
    int get_age() const;
    int get_id() const;
};

#endif
```

Only declarations of the member functions. And the classic way to handle a `static` member: declare it in the class, and define it once in the `.cpp` file.

3.3 `UniPerson`: the implementation

Listing 8 `uni-program/uniperson.cpp`

```cpp
#include "uniperson.h"

int UniPerson::next_id{100000}; // the one definition of the counter

UniPerson::UniPerson(const std::string& name, int age)
    : name{name}, id{next_id++}, age{age} {}

const std::string& UniPerson::get_name() const { return name; }
int UniPerson::get_age() const { return age; }
int UniPerson::get_id() const { return id; }
```

- Outside the class, every member's name gets the prefix `UniPerson::`.
- `int UniPerson::next_id{100000};` is the **one** definition of the static member. If it were in the header, every `.cpp` file including the header would define it again: a link error.

3.4 `Lecturer`: header and implementation

Listing 9 `uni-program/lecturer.h`

```cpp
#ifndef LECTURER_H
#define LECTURER_H

#include "uniperson.h"

class Lecturer : public UniPerson {
  private:
    double salary{};

  public:
    Lecturer(const std::string& name = "", int age = 0, double salary = 42000);
    double get_salary() const;
};

#endif
```

Listing 10 `uni-program/lecturer.cpp`

```cpp
#include "lecturer.h"

Lecturer::Lecturer(const std::string& name, int age, double salary)
    : UniPerson{name, age}, salary{salary} {}

double Lecturer::get_salary() const { return salary; }
```

3.5 `Student`: header and implementation

Listing 11 `uni-program/student.h`

```
#ifndef STUDENT_H
#define STUDENT_H

#include "uniperson.h"

class Student : public UniPerson {
  private:
    double tuition{};

  public:
    Student(const std::string& name = "", int age = 0, double tuition = 9250);
    double get_tuition() const;
};

#endif
```

Listing 12 `uni-program/student.cpp`

```
#include "student.h"

Student::Student(const std::string& name, int age, double tuition)
    : UniPerson{name, age}, tuition{tuition} {}

double Student::get_tuition() const { return tuition; }
```

3.6 `UGStudent`: an inheritance chain

Listing 13 `uni-program/ugstudent.h`

```
#ifndef UGSTUDENT_H
#define UGSTUDENT_H

#include "student.h"

class UGStudent : public Student { // a chain: UniPerson <- Student <- UGStudent
  public:
    UGStudent(const std::string& name = "", int age = 0, double tuition = 9250);
    void write_bsc_thesis() const;
};

#endif
```

Listing 14 uni-program/ugstudent.cpp

```cpp
#include "ugstudent.h"
#include <iostream>

// Default arguments go in the header only (Chapter 05)
UGStudent::UGStudent(const std::string& name, int age, double tuition)
    : Student{name, age, tuition} {}

void UGStudent::write_bsc_thesis() const {
    std::cout << get_name() << " (ID " << get_id()
              << ") is writing a BSc thesis\n";
}
```

3.7 The main program, and building it

Listing 15 uni-program/main.cpp

```cpp
#include "lecturer.h"
#include "ugstudent.h"
#include <iostream>

int main() {
    Lecturer khiem{"Khiem", 42, 55000};
    std::cout << khiem.get_name() << " (ID " << khiem.get_id() << "), salary "
              << khiem.get_salary() << '\n';

    UGStudent natasha{"Natasha", 20};
    std::cout << natasha.get_name() << " (ID " << natasha.get_id()
              << "), tuition " << natasha.get_tuition() << '\n';
    natasha.write_bsc_thesis();
    return 0;
}
```

```
$ g++ -std=c++20 -Wall main.cpp uniperson.cpp lecturer.cpp student.cpp ugstudent.cpp -o uni
$ ./uni
Khiem (ID 100000), salary 55000
Natasha (ID 100001), tuition 9250
Natasha (ID 100001) is writing a BSc thesis
```

4 Order of construction

4.1 A base class and a derived class

Listing 16 `base_and_derived.cpp`

```cpp
class Base {
  public:
    int id{};

    Base(int id = 0) : id{id} {}
    int get_id() const { return id; }
};

class Derived : public Base { // Derived inherits id and get_id()
  public:
    double value{};

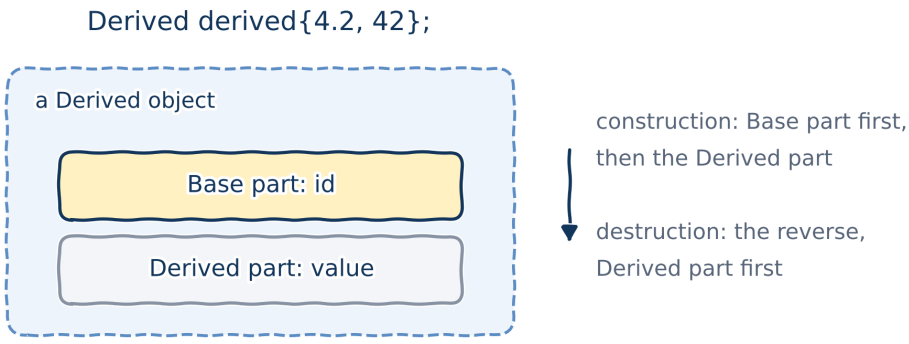
    Derived(double value = 0.0) : value{value} {}
    double get_value() const { return value; }
};

int main() {
    Base base{42};
    Derived derived{42.24};
    return 0;
}
```

`Derived` has its own member `value`, and inherits `id` and `get_id()` from `Base`.

What exactly happens when we create `derived`?

4.2 A derived object has two parts



The members of `Base` are **not copied** into `Derived`. Instead, a `Derived` object is made of two parts: a `Base` part, and a `Derived` part on top of it.

4.3 The base part is built first

Listing 17 `parent_and_child_with_main.cpp`

```cpp
#include <iostream>

class Base {
  public:
    int id{};
    Base(int id = 0) : id{id} { std::cout << "  Base constructed\n"; }
};

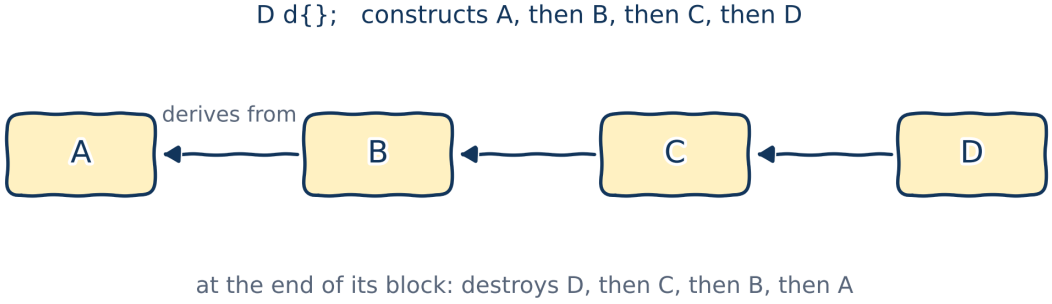
class Derived : public Base {
  public:
    double value{};
    Derived(double value = 0.0) : value{value} {
        std::cout << "  Derived constructed\n";
    }
};

int main() {
    std::cout << "Creating a Base:\n";
    Base base{};
    std::cout << "Creating a Derived:\n";
    Derived derived{};
    return 0;
}
```

```
Creating a Base:
  Base constructed
Creating a Derived:
  Base constructed
  Derived constructed
```

Creating a `Derived` runs the `Base` constructor **first**, then the `Derived` one. It makes sense: the derived part may rely on the base part being ready.

4.4 Inheritance chains



Question: if I create an object of class `D`, which constructors run, and in which order?

4.5 Construction order in a chain

Listing 18 `inheritance_chain_four_classes.cpp`

```cpp
#include <iostream>

class A {
  public:
    A() { std::cout << "  A constructed\n"; }
};

class B : public A {
  public:
    B() { std::cout << "  B constructed\n"; }
};

class C : public B {
  public:
    C() { std::cout << "  C constructed\n"; }
};

class D : public C {
  public:
    D() { std::cout << "  D constructed\n"; }
};

int main() {
    std::cout << "Creating a D:\n";
    D d{};
    return 0;
}
```

```
Creating a D:
  A constructed
  B constructed
  C constructed
  D constructed
```

From the most basic class to the most derived: the great-grandparent `A`, then the grandparent `B`, then the parent `C`, and finally `D` itself.

5 Constructing derived classes

5.1 Coming from Python: `super()`

Listing 19 `uniperson_super.py`

```python
class UniPerson:
    next_id = 100000  # a class variable, like a static member in C++

    def __init__(self, name="", age=18):
        self.name, self.age = name, age
        self.id = UniPerson.next_id
        UniPerson.next_id += 1


class Lecturer(UniPerson):
    def __init__(self, name="", age=18):
        super().__init__(name, age)  # run the parent's constructor first
        print(f"Lecturer {self.name} is recruited.")


khiem = Lecturer("Khiem", 42)
print(f"ID assigned to {khiem.name}: {khiem.id}")
```

```
Lecturer Khiem is recruited.
ID assigned to Khiem: 100000
```

In Python, `super().__init__(...)` runs the parent's constructor. How do we do that in C++?

5.2 The problem

Listing 20 `initialization_of_derived_class.cpp`

```cpp
class Base {
  public:
    int id{};
    Base(int id = 0) : id{id} {}
};

class Derived : public Base {
  public:
    double value{};
    Derived(double value = 0.0) : value{value} {} // and id?
};

int main() {
    Base base{42};         // Base(int): sets id
    Derived derived{4.2}; // Derived(double): sets value... but id?
    return 0;
}
```

`Derived(double)` sets `value`. But how can it set the `id` that every `Derived` inherits from `Base`?

5.3 What happens when a `Derived` is created

When `Base base{42};` runs:

1. memory for a `Base` is set aside;
2. the right `Base` constructor is called: its member initializer list initializes the members, then its body runs.

When `Derived derived{4.2};` runs:

1. memory for a whole `Derived` (both parts) is set aside;
2. the right `Derived` constructor is called, **but first, the `Base` part is constructed** with a `Base` constructor. If we don't say which one, its **default constructor** is used;
3. then `Derived`'s member initializer list and body run.

5.4 Attempt 1: initialize `id` in `Derived`'s list

Listing 21 `initialization_of_derived_class_approach_01.cpp`

```cpp
class Base {
  public:
    int id{};
    Base(int id = 0) : id{id} {}
};

class Derived : public Base {
  public:
    double value{};
    Derived(double value = 0.0, int id = 0)
        : value{value}, id{id} // error: id is not Derived's own member
    {}
};

int main() {
    Derived derived{4.2, 42};
    return 0;
}
```

```
error: class 'Derived' does not have any field named 'id'
```

A member can only be initialized in the member initializer list of **its own** class's constructor. And by then, the `Base` part (with `id`) has already been constructed.

5.5 Attempt 2: assign `id` in the body

Listing 22 `initialization_of_derived_class_approach_02.cpp`

```cpp
class Base {
  public:
    const int id{}; // a const member: try removing const
    Base(int id = 0) : id{id} {}
};

class Derived : public Base {
  public:
    double value{};
    Derived(double value = 0.0, int id = 0) : value{value} {
        this->id = id; // error: assigning to a const member
    }
};

int main() {
    Derived derived{4.2, 42};
    return 0;
}
```

```
error: assignment of read-only member 'Base::id'
```

Remove the `const` and it compiles. But it doesn't work for `const` or reference members, and it's wasteful: `id` is initialized once by `Base`, then assigned again.

5.6 The solution: call the base constructor

Listing 23 `initialization_of_derived_class_base_constructor.cpp`

```cpp
#include <iostream>

class Base {
  public:
    const int id{}; // const is fine now
    Base(int id = 0) : id{id} {}
};

class Derived : public Base {
  public:
    double value{};
    Derived(double value = 0.0, int id = 0)
        : Base{id}, value{value} // call the Base constructor first
    {}
};

int main() {
    Derived derived{4.2, 42};
    std::cout << "id: " << derived.id << ", value: " << derived.value << '\n';
    return 0;
}
```

```
id: 42, value: 4.2
```

Call the `Base` constructor **first** in the member initializer list. It's C++'s `super().__init__()`.

5.7 Passing values up a chain

Listing 24 `initializing_base_members_inheritance_chain.cpp`

```cpp
#include <iostream>

class A {
  public:
    A(int a) { std::cout << "A: " << a << '\n'; }
};

class B : public A {
  public:
    B(int a, double b) : A{a} { std::cout << "B: " << b << '\n'; }
};

class C : public B {
  public:
    C(int a, double b, char c) : B{a, b} { std::cout << "C: " << c << '\n'; }
};

int main() {
    C c{5, 4.3, 'R'}; // C passes a and b up to B; B passes a up to A
    return 0;
}
```

```
A: 5
B: 4.3
C: R
```

Each class calls only its **direct** parent's constructor, passing along what that parent needs.

5.8 Destruction: the reverse order

Listing 25 `destructor_inheritance_chain.cpp`

```cpp
#include <iostream>

class A {
  public:
    A() { std::cout << "A constructed\n"; }
    ~A() { std::cout << "A destroyed\n"; }
};

class B : public A {
  public:
    B() { std::cout << "B constructed\n"; }
    ~B() { std::cout << "B destroyed\n"; }
};

class C : public B {
  public:
    C() { std::cout << "C constructed\n"; }
    ~C() { std::cout << "C destroyed\n"; }
};

int main() {
    C c{};
    return 0;
} // c is destroyed here
```

```
A constructed
B constructed
C constructed
C destroyed
B destroyed
A destroyed
```

When a derived object is destroyed, the destructors run in **reverse** order of construction: the most derived part is taken down first.

6 Inheritance and access specifiers

6.1 The `protected` access specifier

We've used `public` and `private`. The third access specifier, `protected`, only matters with inheritance:

- a **`protected`** member can be used by the class itself, its friends, **and its derived classes**;
- like a `private` member, it can't be used by the public.

Remember the table from Chapter 11? This is its middle column.

6.2 Access specifiers and derived classes

Listing 26 `access_specifiers_in_inheritance.cpp`

```cpp
class Base {
  public:
    int public_member{}; // anyone
  protected:
    int protected_member{}; // Base, its friends, derived classes
  private:
    int private_member{}; // Base and its friends only
};

class Derived : public Base {
  public:
    Derived() {
        public_member = 1;    // fine
        protected_member = 2; // fine: Derived is a derived class
        private_member = 3;   // error: private to Base
    }
};

int main() {
    Base base{};
    base.public_member = 1;    // fine
    base.protected_member = 2; // error: main() is "the public"
    return 0;
}
```

```
error: 'int Base::private_member' is private within this context
error: 'int Base::protected_member' is protected within this context
```

`Derived` can use `protected_member`, but not `private_member`. `main()` can use neither.

6.3 When should we use `protected`?

With a `protected` member, every derived class can use it directly. So if we later change it (its type, its meaning, its name), we may have to change **every** derived class too.

`protected` makes sense when:

- we (or our team) are the ones writing the derived classes;
- there are only a few of them.

In open-source code, anyone can derive from your class and misuse its `protected` members.

> Best practice
>
> Favour `private` members over `protected` ones. Use `protected` only when derived classes are planned, and a `private` member plus public functions would cost too much.

6.4 Kinds of inheritance (read at home)

There are three ways to inherit: `public`, `protected` and `private`. Only `public` is common; the others are rare.

```
class Pub : public Base {};    // what we've used all along
class Pro : protected Base {};
class Pri : private Base {};
class Def : Base {};           // defaults to private for a class!
```

The type of inheritance can make inherited members *less* accessible:

Member in `Base`	`public` inheritance	`protected` inheritance	`private` inheritance
`public`	public	protected	private
`protected`	protected	protected	private
`private`	inaccessible	inaccessible	inaccessible

7 Adding, redefining and extending behaviour

7.1 Adding new functionality

Listing 27 `base_derived.h`

```
#ifndef BASE_DERIVED_H
#define BASE_DERIVED_H

#include <iostream>

class Base {
  protected:
    int id{};

  public:
    Base(int id = 0) : id{id} {}
    void identify() const { std::cout << "I am a Base, ID " << id << '\n'; }
};

class Derived : public Base {
  private:
    double value{}; // new in Derived

  public:
    Derived(double value = 0.0, int id = 0) : Base{id}, value{value} {}
    void print() const { // new in Derived
        std::cout << "ID " << id << ", value " << value << '\n';
    }
};

#endif
```

7.2 Using the new functionality

Listing 28 `add_functionality_example.cpp`

```cpp
#include "base_derived.h"

int main() {
    Derived derived{24.0, 42};
    derived.identify(); // inherited from Base
    derived.print();    // added by Derived
    return 0;
}
```

```
I am a Base, ID 42
ID 42, value 24
```

One of the biggest benefits of inheritance: **reuse** what's written, and **add** what's new. A `Derived` object can use both what it inherited (`identify()`) and what it added (`print()`).

7.3 A vector of derived objects

Listing 29 `add_functionality_example_vector.cpp`

```cpp
#include "base_derived.h"
#include <vector>

int main() {
    std::vector<Derived> list{Derived{42.0, 24}, Derived{24.0, 42}, Derived{}};

    for (const auto& element : list) { // like "for element in list:"
        element.print();
    }
    return 0;
}
```

```
ID 24, value 42
ID 42, value 24
ID 0, value 0
```

`for (const auto& element : list)` is a **range-based for loop**: it visits every element of `list`, like Python's `for element in list:`. The `const auto&` avoids copying each element (Chapter 07).

7.4 Calling a base class function

Listing 30 `call_base_class_function.cpp`

```cpp
#include <iostream>

class Base {
  public:
    void identify() const { std::cout << "Base::identify()\n"; }
};

class Derived : public Base {}; // nothing of its own

int main() {
    Base base{};
    base.identify();
    Derived derived{};
    derived.identify(); // the inherited Base::identify()
    return 0;
}
```

```
Base::identify()
Base::identify()
```

`Derived` inherits `identify()` from `Base`, so `derived.identify()` calls `Base::identify()`.

7.5 Redefining behaviour

Listing 31 `redefine_behaviors.cpp`

```cpp
#include <iostream>

class Base {
  public:
    void identify() const { std::cout << "Base::identify()\n"; }
};

class Derived : public Base {
  public:
    void identify() const { std::cout << "Derived::identify()\n"; }
};

int main() {
    Base base{};
    base.identify();
    Derived derived{};
    derived.identify(); // Derived's own version hides Base's
    return 0;
}
```

```
Base::identify()
Derived::identify()
```

If `Derived` defines its own `identify()`, a `Derived` object uses that one instead: it **hides** the base version.

7.6 Adding to existing behaviour

Listing 32 `add_to_existing_functionality.cpp`

```cpp
#include <iostream>

class Base {
  public:
    void identify() const { std::cout << "Base::identify()\n"; }
};

class Derived : public Base {
  public:
    void identify() const {
        std::cout << "Derived::identify()\n";
        Base::identify(); // ...and then do what Base does
    }
};

int main() {
    Derived derived{};
    derived.identify();
    return 0;
}
```

```
Derived::identify()
Base::identify()
```

To *extend* the base behaviour rather than replace it, call the base version explicitly: `Base::identify()`.

7.7 Changing access with `using` (read at home)

Listing 33 `change_inherited_member_access_level.cpp`

```cpp
#include <iostream>

class Base {
  private:
    int id{};

  public:
    Base(int id) : id{id} {}

  protected:
    void print_id() const { std::cout << "ID " << id << '\n'; }
};

class Derived : public Base {
  public:
    Derived(int id) : Base{id} {}
    using Base::print_id; // protected in Base, public in Derived
};

int main() {
    Derived d{42};
    d.print_id(); // fine now
    return 0;
}
```

```
ID 42
```

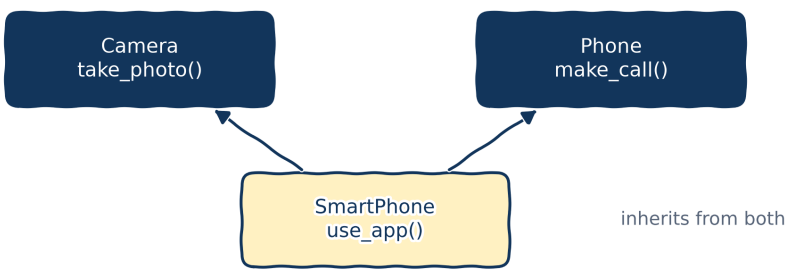
A **using declaration** in the derived class changes an inherited member's access level. Under `public:`, `using Base::print_id;` makes a `protected` function public.

Under `private:`, the same trick **hides** a public member (`hide_functionality.cpp`). And `= delete` forbids calling it at all (`delete_functions_in_derived.cpp`).

8 Multiple inheritance

8.1 Multiple inheritance

Multiple inheritance lets a class inherit from more than one base class.



A `Camera` can `take_photo()`; a `Phone` can `make_call()`; a `SmartPhone` is both, and adds apps on top.

8.2 A smartphone in code

Listing 34 `multiple_inheritance_smartphone.cpp`

```cpp
#include <iostream>
#include <string>

class Camera {
  public:
    void take_photo() const { std::cout << "Taking a photo!\n"; }
};

class Phone {
  public:
    void make_call() const { std::cout << "Making a phone call!\n"; }
};

class SmartPhone : public Camera, public Phone { // two base classes
  public:
    void use_app(const std::string& app) const {
        std::cout << "Using " << app << "!\n";
    }
};

int main() {
    SmartPhone my_phone{};
    my_phone.take_photo(); // from Camera
    my_phone.make_call();  // from Phone
    my_phone.use_app("Moodle");
    return 0;
}
```

```
Taking a photo!
Making a phone call!
Using Moodle!
```

> **Watch out**
>
> Multiple inheritance gets complicated fast: what if both bases have a member with the same name? Use it only when the alternatives are worse.

9 Wrapping up

9.1 Summary

- **Inheritance** models "is a": `class Lecturer : public UniPerson`. The derived class reuses the base class's members and adds its own.
- A derived object has a **base part** and a **derived part**. The base part is constructed first and destroyed last; in a chain, from the most basic class to the most derived.
- Initialize the base part by calling the **base constructor** first in the member initializer list: C++'s `super().__init__()`.

- `protected` members are visible to derived classes, but not to the public. Prefer `private`.
- A derived class can **add** functions, **redefine** (hide) inherited ones, or **extend** them by calling `Base::function()`.
- A class can inherit from **several** bases, but use that sparingly.

9.2 Keep learning

Explore these on your own, with Google Gemini in Guided Learning mode or on learncpp.com:

1. Constructors and initialization of derived classes
2. Inheritance and access specifiers
3. Hiding inherited functionality
4. Multiple inheritance

Next up, Chapter 15: virtual functions and polymorphism.